

Manual de Referencia: Profiling y Optimización Multihilo de Simuladores Físicos en Python

Guía Técnica de Desarrollo

30 de julio de 2026

Índice

1. Evaluación y Profiling del Tiempo de Ejecución	2
1.1. Medición Directa por Bloques con <code>time.perf_counter()</code>	2
1.2. Profiling Global con <code>cProfile</code> y Visualización	2
2. Estrategias para Desacoplar Renderizado e I/O	3
2.1. Arquitectura Multihilo y Colas Thread-Safe	3
2.2. Optimización de Disco y Gráficos	3
3. Implementación Completa: Ejemplo Integrado	4

1. Evaluación y Profiling del Tiempo de Ejecución

Cuando un simulador físico escrito en Python incluye un cálculo numérico (resolución de ODEs), una interfaz gráfica (GUI) y almacenamiento en disco (I/O), es fundamental medir qué porcentaje del tiempo total consume cada componente para detectar cuellos de botella.

1.1. Medición Directa por Bloques con `time.perf_counter()`

La forma más precisa para aislar estos bloques dentro del bucle de integración principal consiste en medir el tiempo acumulado con un temporizador de alta resolución del sistema operativo.

```
1 import time
2
3 # Acumuladores de tiempo total (en segundos)
4 t_sim, t_gui, t_io = 0.0, 0.0, 0.0
5 n_pasos = 0
6
7 def paso_de_simulacion():
8     global t_sim, t_gui, t_io, n_pasos
9
10    # 1. C lculo Num rico
11    t0 = time.perf_counter()
12    estado_nuevo = resolver_ode_paso(estado_actual)
13    t1 = time.perf_counter()
14    t_sim += (t1 - t0)
15
16    # 2. Escritura en Disco
17    t0 = time.perf_counter()
18    escribir_archivo(estado_nuevo)
19    t1 = time.perf_counter()
20    t_io += (t1 - t0)
21
22    # 3. Actualizaci n de Interfaz / Gr fico
23    t0 = time.perf_counter()
24    actualizar_grafico_gui(estado_nuevo)
25    t1 = time.perf_counter()
26    t_gui += (t1 - t0)
27
28    n_pasos += 1
29
30 def mostrar_reporte():
31     t_total = t_sim + t_gui + t_io
32     print(f"--- REPORTE DE RENDIMIENTO ({n_pasos} pasos) ---")
33     print(f"Simulaci n (C lculo): {t_sim:.4f} s ({t_sim/t_total*100:.1f}%)")
34     print(f"Escritura en Disco: {t_io:.4f} s ({t_io/t_total*100:.1f}%)")
35     print(f"Interfaz Gr fica: {t_gui:.4f} s ({t_gui/t_total*100:.1f}%)")
36     print(f"Tiempo Total Medido: {t_total:.4f} s")
```

Listing 1: Medición del tiempo por bloques en el bucle principal.

1.2. Profiling Global con `cProfile` y Visualización

Para inspeccionar de manera interna qué llamadas dentro de bibliotecas como Matplotlib, Scipy o la GUI consumen recursos, se ejecuta:

```

1 # Generar el archivo de profiling
2 python -m cProfile -o resultado.prof tu_programa.py
3
4 # Visualizar interactivamente en el navegador
5 pip install snakeviz
6 snakeviz resultado.prof

```

Listing 2: Uso de cProfile y SnakeViz desde la terminal.

2. Estrategias para Desacoplar Renderizado e I/O

El principio clave para evitar que la GUI y la escritura en disco frenen la simulación es **desacoplar los tiempos de cada proceso**.

2.1. Arquitectura Multihilo y Colas Thread-Safe

- **Hilo de Simulación:** Resuelve las ecuaciones diferenciales en un bucle continuo sin pausas.
- **Hilo Principal (GUI):** Mantiene la respuesta de la interfaz a 60 FPS consumiendo solo estados recientes.
- **Hilo de Disco (I/O):** Recibe el 100 % de las muestras y las escribe en bloques.
- **Colas (queue.Queue):** Permiten la comunicación no bloqueante (`put_nowait()`).

2.2. Optimización de Disco y Gráficos

1. **Escritura en Bloques (*Batching*):** Acumular datos en memoria y volcarlos a disco cada N iteraciones mediante `writerows()` para minimizar las llamadas al sistema.
2. **Subsampling (Decimación):** Reducir la tasa de actualización de la GUI (ej. enviar a la cola 1 de cada 10 pasos).
3. **Blitting en Matplotlib:** Redibujar únicamente la línea modificada en lugar de recalculando ejes, grillas y etiquetas en cada cuadro.

Cuadro 1: Resumen de estrategias de optimización.

Técnica	Problema que Resuelve	Impacto Esperado
Multihilo + Queue	Congelamiento de la simulación	Alto (solver independiente)
<i>Batching</i> en I/O	Latencia por llamadas a sistema	Reduce el impacto de I/O a <1 %
Blitting / PyQtGraph	Sobrecarga de renderizado de la GPU/CPU	Muy alto (fluidez visual a 60 FPS)

3. Implementación Completa: Ejemplo Integrado

El siguiente código implementa un simulador completo en **PyQt6/PyQt5** utilizando tres hilos de ejecución concurrentes:

1. `SimulationWorker` (Cálculo numérico).
2. `DiskWorker` (Escritura en disco con *batching*).
3. `MainWindow` (Hilo principal de la GUI con *Blitting* en Matplotlib).

```
1 import sys
2 import time
3 import csv
4 import numpy as np
5 from queue import Queue, Empty
6
7 from PyQt6.QtCore import QThread, QTimer
8 from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget,
9                               QVBoxLayout, QHBoxLayout, QPushButton)
10
11 from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as
    FigureCanvas
12 from matplotlib.figure import Figure
13
14 #
15 # =====
16 # 1. HILO DE ESCRITURA EN DISCO (I/O Worker con Batching)
17 # =====
18
19 class DiskWorker(QThread):
20     def __init__(self, disk_queue: Queue, filename="
simulacion_resultados.csv", batch_size=1000):
21         super().__init__()
22         self.disk_queue = disk_queue
23         self.filename = filename
24         self.batch_size = batch_size
25         self.running = False
26
27     def run(self):
28         self.running = True
29         buffer = []
30
31         with open(self.filename, mode="w", newline="", encoding="utf-8")
as f:
32             writer = csv.writer(f)
33             writer.writerow(["Tiempo_s", "Posicion_x", "Velocidad_v"])
34
35             while self.running or not self.disk_queue.empty():
36                 try:
37                     data = self.disk_queue.get(timeout=0.1)
38                     buffer.append(data)
39                     self.disk_queue.task_done()
40
41                     # Volcado masivo a disco al acumular N elementos
42                     if len(buffer) >= self.batch_size:
```

```

41         writer.writerow(buffer)
42         buffer.clear()
43
44         except Empty:
45             continue
46
47         if buffer:
48             writer.writerow(buffer)
49             buffer.clear()
50
51     def stop(self):
52         self.running = False
53         self.wait()
54
55 #
56 # =====
57 # 2. HILO DE SIMULACION (Cálculo Numérico Puro)
58 # =====
59
60 class SimulationWorker(QThread):
61     def __init__(self, gui_queue: Queue, disk_queue: Queue):
62         super().__init__()
63         self.gui_queue = gui_queue
64         self.disk_queue = disk_queue
65         self.running = False
66
67     def run(self):
68         self.running = True
69
70         t = 0.0
71         dt = 0.001
72         x = 1.0
73         v = 0.0
74         gamma = 0.1
75         omega0 = 2.0 * np.pi
76
77         step_counter = 0
78
79         while self.running:
80             # Integración numérica (Euler-Cromer)
81             a = -2 * gamma * v - (omega0**2) * x
82             v += a * dt
83             x += v * dt
84             t += dt
85
86             step_counter += 1
87
88             # 1. ENVIAR A DISCO: Se guardan TODOS los pasos
89             try:
90                 self.disk_queue.put_nowait((t, x, v))
91             except Exception:
92                 pass
93
94             # 2. ENVIAR A GUI: Subsampling (1 de cada 10 pasos)
95             if step_counter % 10 == 0:
96                 try:

```

```

95         self.gui_queue.put_nowait((t, x))
96     except Exception:
97         pass
98
99         time.sleep(0.0001)
100
101     def stop(self):
102         self.running = False
103         self.wait()
104
105 #
=====
106 # 3. VENTANA PRINCIPAL (GUI y Renderizado con Blitting)
107 #
=====
108 class MainWindow(QMainWindow):
109     def __init__(self):
110         super().__init__()
111         self.setWindowTitle("Simulador Multihilo (C lculo + GUI + Disco
112         )")
113
114         self.resize(800, 500)
115
116         self.gui_queue = Queue(maxsize=100)
117         self.disk_queue = Queue(maxsize=50000)
118
119         self.sim_worker = SimulationWorker(self.gui_queue, self.
120         disk_queue)
121         self.disk_worker = DiskWorker(self.disk_queue)
122
123         main_widget = QWidget()
124         self.setCentralWidget(main_widget)
125         layout = QHBoxLayout(main_widget)
126
127         controls_layout = QVBoxLayout()
128         self.btn_start = QPushButton("Iniciar Simulaci n")
129         self.btn_start.clicked.connect(self.toggle_simulation)
130         controls_layout.addWidget(self.btn_start)
131         controls_layout.addStretch()
132         layout.addLayout(controls_layout)
133
134         self.fig = Figure(figsize=(6, 4))
135         self.canvas = FigureCanvas(self.fig)
136         layout.addWidget(self.canvas)
137
138         self.ax = self.fig.add_subplot(111)
139         self.ax.set_xlim(0, 10)
140         self.ax.set_ylim(-1.5, 1.5)
141         self.ax.set_xlabel("Tiempo (s)")
142         self.ax.set_ylabel("Posici n (x)")
143         self.ax.grid(True)
144
145         (self.line,) = self.ax.plot([], [], 'b-', animated=True)
146
147         self.t_data = []
148         self.x_data = []
149         self.background = None

```

```

147
148     self.canvas.mpl_connect("draw_event", self.on_draw)
149
150     self.timer = QTimer()
151     self.timer.setInterval(16) # ~60 FPS
152     self.timer.timeout.connect(self.update_gui)
153
154     def on_draw(self, event):
155         self.background = self.canvas.copy_from_bbox(self.ax.bbox)
156         self.ax.draw_artist(self.line)
157
158     def toggle_simulation(self):
159         if not self.sim_worker.isRunning():
160             self.t_data.clear()
161             self.x_data.clear()
162
163             self.disk_worker.start()
164             self.sim_worker.start()
165             self.timer.start()
166
167             self.btn_start.setText("Detener Simulaci n")
168         else:
169             self.sim_worker.stop()
170             self.disk_worker.stop()
171             self.timer.stop()
172
173             self.btn_start.setText("Iniciar Simulaci n")
174
175     def update_gui(self):
176         if self.background is None:
177             return
178
179         updated = False
180         while True:
181             try:
182                 t, x = self.gui_queue.get_nowait()
183                 self.t_data.append(t)
184                 self.x_data.append(x)
185                 updated = True
186             except Empty:
187                 break
188
189         if updated:
190             if self.t_data[-1] > 10:
191                 self.ax.set_xlim(self.t_data[-1] - 10, self.t_data[-1])
192                 self.canvas.draw()
193                 return
194
195         # Renderizado Blitting
196         self.line.set_data(self.t_data, self.x_data)
197         self.canvas.restore_region(self.background)
198         self.ax.draw_artist(self.line)
199         self.canvas.blit(self.ax.bbox)
200
201     def closeEvent(self, event):
202         self.sim_worker.stop()
203         self.disk_worker.stop()
204         event.accept()

```

```

205
206 #
    =====
207 # 4. PUNTO DE ENTRADA
208 #
    =====

209 if __name__ == "__main__":
210     app = QApplication(sys.argv)
211     window = MainWindow()
212     window.show()
213     sys.exit(app.exec())

```

Listing 3: Código completo del simulador desacoplado multihilo.